

فيما يلي 100 Code Challenges مبنية على 3 مستويات ومتوافقة مع خارطة
“Mastering NodeJS from Basics to Advanced Levels”.
تم ترتيبها بحيث تكون الأسئلة متوافقة مع كل مستوى كل تحدٍ قصير وعملي، لتناسب
الدروس المجدولة مسبقا ، مع تغطية شاملة للموضوعات.

—NODEJS Challenges (Questions Only)

Level 1 — NodeJS Essentials (1–30)

1. Print & CLI args

Write a script that prints `Hello` followed by all CLI args joined by `-`. Example: `node app.js a b` → `Hello-a-b`.

2. CommonJS export

Create `math.js` exporting `add(a,b)` and use it from `index.js` to log 7 for `add(3,4)`.

3. ESM import

Convert the previous challenge to ESM (`type:"module"`) and import `add` via `import`.

4. process.env

Read `PORT` from `env` with fallback `3000` and log `Listening on :PORT`.

5. Event loop order #1

What's the output order?

```
console.log('A');
setTimeout(()=>console.log('B'),0);
Promise.resolve().then(()=>console.log('C'));
process.nextTick(()=>console.log('D'));
console.log('E');
```

6. Event loop order #2

Guarantee that `after-read` logs **after** an async `fs.readFile`.
Where do you place `setImmediate(()=>console.log('after-read'))`?

7. **fs readFile vs readFileSync**

Read `data.txt` and log bytes count twice: once sync, once async.

8. **fs write & append**

Create `out.txt` with `Hello`, then append `World`.

9. **fs/promises**

Using `await`, write JSON `{ok:true}` to `result.json`.

10. **path.join**

Given `__dirname='/usr/app'`, join `assets`, `img`, `logo.png`. Print normalized path.

11. **url parsing**

Parse `https://example.com/api?x=10&y=test` and log `x` and `y`.

12. **EventEmitter basics**

Create emitter, register `on('ping')`, emit twice, count calls.

13. **Remove listener**

Add two listeners `log1`, `log2`; remove `log2`; prove only `log1` runs.

14. **HTTP server (core)**

Create server on 3000 returning `{"status":"ok"}` with `Content-Type: application/json`.

15. **HTTP routing (core)**

Return 200 `/hello` → `Hello`, and 404 otherwise.

16. **Stream readable**

Create readable stream from `input.txt` and count number of lines.

17. **Stream pipe**

Pipe `big.txt` through `gzip` to `big.txt.gz`.

18. **Backpressure**

Implement manual `readable.pipe(writable)` replacement handling `drain`.

19. **Timers**

Implement a function `delay(ms)` using `Promise` and `setTimeout`.

20. **NPM scripts**

Add `scripts.start` that runs `node server.js` and `scripts.dev` that runs `node --watch server.js`.

21. **Semantic versioning**
Explain difference between `^1.2.3` and `~1.2.3` in `package.json` with one example.
 22. **npx**
Use `npx` to run a one-off script `cowsay "Node!"`. Show command only.
 23. **OS info**
Use `os` module to print platform, cpus count, and total memory (MB).
 24. **Console timing**
Measure how long `fs.readFile('large.bin')` takes using `console.time`.
 25. **Buffer basics**
Create a `Buffer` from UTF-8 string and print its hex representation.
 26. **Buffer concat**
Concat two buffers `Hello` and `World` → `HelloWorld`.
 27. **Global `__dirname` in ESM**
Emulate `__dirname` in ESM using `import.meta.url`.
 28. **AbortController**
Cancel a long `setTimeout` using `AbortController` + `timers/promises`.
 29. **REPL**
Start Node REPL and evaluate `process.version`. Provide command to enter REPL and how to exit.
 30. **Small CLI**
Create `cli.js` that supports `--upper "text"` and prints it uppercased; default prints help.
-

Level 2 — NodeJS Intermediate (31–70)

31. **Callbacks → Promises**

Convert:

```
fs.readFile('a.txt', (e,d)=>{ if(e) throw e;  
console.log(d.length); });
```

to `fs.promises` with `await`.

32. **Promise.all**

Read two files `a.txt`, `b.txt` concurrently and print total bytes.

33. **Race vs AllSettled**

Explain difference using reading `a.txt` (exists) and `missing.txt` (missing).

34. **Error handling**

Write `safe(asyncFn)` that wraps an async function and returns `[err, data]`.

35. **Retry**

Implement `retry(fn, times)` for a failing async function.

36. **HTTP JSON server (core)**

Build POST `/sum` reading JSON `{a,b}` from body; respond `{sum:a+b}`.

37. **Query params**

For GET `/greet?name=Sara`, respond `Hello, Sara`.

38. **Middleware concept (manual)**

Implement a minimal middleware runner: an array of functions `(req, res, next)` executed in order.

39. **Express: basic app**

Create Express app: GET `/` → OK.

40. **Express: params**

GET `/users/:id` responds `{id}` as number.

41. **Express: middleware**

Add middleware to log method + url + duration.

42. **Express: error handler**
Add centralized error handler returning `{message}` and status 500.
43. **Express: validation**
POST `/users` requires `email`; if missing, respond 400.
44. **Express: static**
Serve static files from `/public` under `/static`.
45. **Express: templating**
Render `index.ejs` with `{title:"Home"}`.
46. **Express: CRUD**
Implement in-memory CRUD for `/todos`.
47. **MongoDB: connect**
Connect with Mongoose to `mongodb://localhost:27017/app` and log connected.
48. **Mongoose: schema**
Create `User{email:String unique, createdAt>Date default:now}` model.
49. **Mongoose: CRUD**
Create, find by email, update, and delete a `User`.
50. **Mongoose: validation**
Add validator: email required & matches pattern.
51. **Mongoose: index**
Create compound index `{email:1, createdAt:-1}`.
52. **Mongo aggregation**
Aggregate `orders` to sum `total` by `status`.
53. **Mongo transactions (session)**
Start a session, insert two docs, commit.
54. **SQL with Prisma: schema**
Define `User(id Int @id @default(autoincrement), email String @unique)`.
55. **Prisma: query**
Find all users with email domain `@example.com`.
56. **Prisma: update**
Update user by id, set `verified:true`.

57. **Security: sanitize**
Given `name = req.query.name`, safely echo it without XSS in Express response.
58. **Rate limiting**
Add simple in-memory rate limit: 60 requests/minute per IP.
59. **Cookie & Session**
Set signed cookie `sid` and read it back.
60. **JWT login**
Upon correct credentials, sign JWT (HS256) with `userId` and return token.
61. **JWT verify middleware**
Middleware that verifies `Authorization: Bearer <token>`.
62. **Password hashing**
Hash password with `bcrypt` and verify on login.
63. **CSRF concept**
Explain how CSRF can affect a cookie-based session and one mitigation in Express.
64. **CORS**
Enable CORS for `https://app.example.com` only.
65. **File upload**
Use `multer` to accept single file `avatar` to `uploads/`.
66. **WebSocket (ws)**
Create ws server; echo back messages prefixed with `server::`.
67. **Socket.IO**
Rooms: join room `room1` and broadcast `message` to that room.
68. **Config per env**
Load `.env` and require `NODE_ENV` to be one of dev/test/prod.
69. **Logger**
Configure `winston` with JSON format and daily rotate file.
70. **Graceful shutdown**
On `SIGINT`, stop HTTP server and close DB connections before exit.

Level 3 — NodeJS Advanced (71–100)

71. **Streams: transform**
Create a transform stream that uppercases all input.
72. **Streams: CSV parse**
Parse CSV lines `name,age` and output JSON lines.
73. **Buffer & encoding**
Convert a base64 string to UTF-8 text and back.
74. **Cluster**
Create a clustered HTTP server using all CPUs; log worker PIDs.
75. **Child process exec**
Run `ls -la` and print stdout length.
76. **Child process spawn**
Spawn `node -v` and pipe stdout to parent process.
77. **Worker Threads**
Run CPU-heavy Fibonacci in a Worker and post result.
78. **MessageChannel**
Use `MessageChannel` to pass data between main thread and worker.
79. **Perf hooks**
Use `performance.mark/measure` around an async operation.
80. **Caching: memory**
Cache GET `/data` for 5 seconds in-memory.
81. **Caching: Redis**
Pseudocode: GET `/posts` checks Redis before DB; set TTL 60s.
82. **Queue (BullMQ)**
Enqueue `email:send` job and process it with concurrency 5.
83. **Microservice HTTP**
Service A calls Service B's `/sum?a=..&b=..`; implement A's client with retries.
84. **GraphQL (Apollo)**
Define type `Query { hello: String }` and resolver returning `Hello`.

- 85. **GraphQL mutation**
Add `Mutation { add(a:Int,b:Int): Int }`.
- 86. **TypeScript: types**
Create `types.d.ts` defining `User` and use it in `app.ts`.
- 87. **TS + Node config**
Minimal `tsconfig.json` for Node ESM target ES2022.
- 88. **Zod validation**
Validate `{email,age}` with zod, return 400 on failure.
- 89. **Feature flags**
Read `FEATURE_NEW_HOME` from env; toggle route response accordingly.
- 90. **OpenAPI (Swagger)**
Add Swagger UI to Express at `/docs`.
- 91. **Testing: Jest unit**
Test `sum(2,3)` returns 5.
- 92. **Testing: Supertest**
Test GET `/health` returns 200 and `{ok:true}`.
- 93. **Mocking**
Mock `fs.readFile` in a test to return `hello`.
- 94. **Debugging**
Show how to run Node with inspector and set a breakpoint in VS Code (launch args).
- 95. **PM2**
Start app with PM2 cluster mode `max` and zero-downtime reload.
- 96. **Env config library**
Use `dotenv + zod` to validate required env vars at startup.
- 97. **Helmet & Security headers**
Enable Helmet and verify `X-Content-Type-Options` is set.
- 98. **Input size limit**
Limit JSON body size to `100kb` in Express.
- 99. **Rate limit with Redis**
Sketch Redis-based token bucket: key per IP, refill rate 1 r/s, burst 10.

100. **Blue/Green deploy concept**

Explain in 3 lines how to do blue/green with two Node app versions behind a load balancer.
